

untangle::actuator

Generated by Doxygen 1.18.0

1 actuator	1
2 Topic Index	5
2.1 Topics	5
3 Namespace Index	7
3.1 Namespace List	7
4 Concept Index	9
4.1 Concepts	9
5 Class Index	11
5.1 Class List	11
6 File Index	13
6.1 File List	13
7 Topic Documentation	15
7.1 namespace untangle: functions	15
7.1.1 Detailed Description	16
7.1.2 Function Documentation	16
7.1.2.1 connect() [1/4]	16
7.1.2.2 connect() [2/4]	17
7.1.2.3 connect() [3/4]	18
7.1.2.4 connect() [4/4]	19
7.1.2.5 bind() [1/3]	19
7.1.2.6 bind() [2/3]	20
7.1.2.7 bind() [3/3]	20
8 Namespace Documentation	23
8.1 untangle Namespace Reference	23
8.1.1 Detailed Description	24
8.1.2 Typedef Documentation	24
8.1.2.1 task_t	24
8.1.3 Function Documentation	25
8.1.3.1 last_arg()	25
8.1.3.2 connect_one()	25
8.1.3.3 connect_named_one()	26
8.1.3.4 bind_task()	27
9 Concept Documentation	29
9.1 untangle::task_callback_for Concept Reference	29
9.1.1 Concept definition	29
9.1.2 Detailed Description	29
9.2 untangle::testable_for_emptiness Concept Reference	30
9.2.1 Concept definition	30

9.2.2 Detailed Description	30
10 Class Documentation	31
10.1 untangle::actuator< action_t > Struct Template Reference	31
10.1.1 Detailed Description	33
10.1.2 Member Typedef Documentation	34
10.1.2.1 actions_t	34
10.1.2.2 tasks_t	34
10.1.2.3 results_t	35
10.1.2.4 errors_t	35
10.1.3 Constructor & Destructor Documentation	35
10.1.3.1 actuator()	35
10.1.4 Member Function Documentation	35
10.1.4.1 operator=()	35
10.1.4.2 type()	36
10.1.4.3 reset()	36
10.1.4.4 copy_from()	36
10.1.4.5 release_owned()	37
10.1.4.6 invoke_callback()	37
10.1.4.7 take_callback()	38
10.1.4.8 operator()()	39
10.1.4.9 invoke_action()	39
10.1.4.10 call_tasks()	40
10.1.4.11 add() [1/4]	40
10.1.4.12 add() [2/4]	41
10.1.4.13 add() [3/4]	41
10.1.4.14 add() [4/4]	41
10.1.4.15 add_task()	42
10.1.4.16 remove() [1/2]	42
10.1.4.17 remove() [2/2]	43
10.1.4.18 is_connected()	44
10.1.4.19 has_tasks()	44
10.1.4.20 has_action()	44
10.1.5 Member Data Documentation	45
10.1.5.1 tasks	45
10.1.5.2 owned	45
10.1.5.3 dispatching	45
10.2 untangle::invalid_action Struct Reference	46
10.2.1 Detailed Description	46
10.2.2 Constructor & Destructor Documentation	46
10.2.2.1 invalid_action()	46
10.2.3 Member Function Documentation	46
10.2.3.1 what()	46

10.3 untangle::actuator< action_t >::no_callback Struct Reference	47
10.3.1 Detailed Description	47
11 File Documentation	49
11.1 actuator.hpp	49
Index	57

Chapter 1

actuator

An actuator is a generic callable that may invoke a list of actions(callables of type `std::function<...>`).

Its goal is to provide a light and simple mechanism to connect components that want to communicate with each other. It works similar as the "signal and slots" mechanisms from other frameworks, but much simplified.

Modern programming languages do not offer class intrinsic interfaces that would improve significantly how objects are connected and how they communicate. Instead, languages, like C++, are using external interfaces, that are not only cluttering the code, but they don't feel "natural" as they are opposed to what an interface is in reality. Basically the objects and interfaces are not separated, and the interface is intrinsic to the object(think on the cogs in a mechanism). A generic callable is an attempt to mitigate this problem. `async` implementation is such use case.

An actuator provides the results of all actions in a vector, in the same order as the actions were added.

Example: how to use actuator instead of polymorphism

```
void rotate_shapes(const std::vector<shape*>& shapes, int angle)
{
    for (const auto& s : shapes)
    {
        s->rotate(angle);
    }
}

std::shared_ptr<triangle> t(new triangle);
std::shared_ptr<circle> c(new circle);
std::shared_ptr<square> s(new square);

// using polymorphism
std::vector<shape*> shapes;
shapes.push_back(t.get());
shapes.push_back(c.get());
shapes.push_back(s.get());

rotate_shapes(shapes, 10);

// using actuator
auto action1 = untangle::bind(t, &triangle::rotate);
auto action2 = untangle::bind(c, &circle::rotate);
auto action3 = untangle::bind(s, &square::rotate);
auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate(20);
```

Anonymous lambdas as actions

An actuator normally does not own its actions: it stores pointers to `std::function` objects the caller keeps alive. `add()` also has an overload that takes an action by value, so an anonymous lambda can be an action with no named variable to keep around, and returns a handle to pass to `remove()`:

```
untangle::actuator<std::function<int(int)>> actuator_scale;
auto* handle = actuator_scale.add([](int v) { return v * 2; });
actuator_scale.add("triple", [](int v) { return v * 3; });
```

`connect()` accepts them too, but it deduces the action type from its arguments, and a lambda has its own closure type – it is not a `std::function` until something converts it. A call made only of anonymous lambdas therefore has nothing to deduce from, and the signature has to be named on `connect()` itself:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    [](int angle) { /* ... */ }, [](int angle) { /* ... */ });
```

Naming the type of the variable the result is assigned to does not supply it – template arguments are deduced from the call arguments alone, never from what the returned value is assigned to:

```
// ill formed: "no matching function for call to connect"
untangle::actuator<std::function<void(int)>> actuator_rotate =
    untangle::connect([](int angle) { /* ... */ });
```

One named action anywhere in the call deduces the type for the whole of it, and the anonymous lambdas beside it then need nothing:

```
auto actuator_rotate = untangle::connect(action1, [](int angle) { /* ... */ });
```

The named form of `connect()` works the same way: a pair holding a pointer names an action the caller owns, a pair holding the action by value hands it over to the actuator, and the two can be mixed. The action type is deduced from the pointer, so a call whose first pair holds a lambda has to name the signature:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    std::make_pair("triangle", [](int angle) { /* ... */ }),
    std::make_pair("circle", [](int angle) { /* ... */ }));

// deduced from the leading pointer, so the lambda beside it needs nothing
auto actuator_mixed = untangle::connect(std::make_pair("triangle", &action1),
    std::make_pair("circle", [](int angle) { /* ... */ }));
```

Tasks: an action bound to its arguments and to the callback it must notify

An action is a subscription: it lives across invocations, and it is fired with a pack the caller supplies each time. A task is a one-shot. It carries its own arguments, is fired once, and reports what it produced to a callback it was built with. The two live side by side in one actuator and share nothing else – `operator()` fires the actions and leaves the tasks alone, `call_tasks()` fires the tasks and leaves the actions alone.

A task exists so that work which runs later than it was described can still say what became of it. That is what a queue needs, and an actuator holding only actions cannot give: the callback convention on `operator()` reads the trailing argument of the invocation, and a queued call has no invocation left to read from.

```
untangle::actuator<std::function<int(int)>> actuator_scale;

// the callback comes last, after the arguments the action is bound to
actuator_scale.add_task(untangle::bind_task(action, 21, [](int result) { /* result == 42 */ }));

actuator_scale.call_tasks(); // runs it, notifies it, and empties the list
```

A task without a callback does not exist. That is the whole difference from an action's callback, which is optional and is recognised by its type – a trailing argument that does not look like a callback is simply an argument, and nothing fires. A task's callback is the last argument of `bind_task()`, taken by position, and `task_callback_for` only checks that it can serve. There is no answer "no" to fall through to, so a missing or unusable callback is a `static_assert` rather than silence.

It is not forwarded to the action either, so an action needs no callback parameter of its own – and a task whose action returns void is a task like any other. It reports with a void() callback, because finished is the message and the result is optional:

```
untangle::actuator<std::function<void(int)>> actuator_notify;
actuator_notify.add_task(untangle::bind_task(action, 5, [] { /* finished */ }));
```

The arguments are copied when the task is built, while the caller still holds them, and handed to the action as the task's own lvalues when it runs. A task outlives the call that described it and cannot borrow what may already be gone; a move-only argument therefore does not compile.

add_task() answers bool. It refuses a task that has nothing to run or no callback to notify, and the refusal is the whole report – nothing is thrown and nothing is stored. Both are the caller's own mistake, and both are caught while the caller is still on the stack rather than surfacing later from call_tasks() as a std::bad_function_call recorded against a task whose action had in fact succeeded.

Finished does not mean failed. A task whose action throws is not notified: there is no result to hand over, and for a void task no completion to report either. What it threw goes to actuator::errors, the tasks behind it still run, and a caller relying on the callback alone will never hear about it. A callback runs inside the same try as the task that owns it, so what it throws travels the same path – errors can hold a failure for a task that succeeded.

A task's result goes to its callback and nowhere else: actuator::results is how an action hands back what it returned, and a task was built with something better. call_tasks() appends to errors rather than clearing it, while operator() clears both as it starts – so an actuator fired as one(); one.call_tasks(); reports both kinds together, and the actions go first.

Ask has_tasks() whether a task is waiting. is_connected() answers for the actions and for nothing else: an actuator holding only tasks is not connected, and a queue reading the wrong one would report a batch of tasks as nothing to do.

For convenience there are provided helpers methods to "connect" to an initial list of "actions", or to create bindings to class methods.

Please check the manual in doc/refman.pdf for further references.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [15](#)

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	Interface to untangle::actuator functor	23
--------------------------	---	--------------------

Chapter 4

Concept Index

4.1 Concepts

Here is a list of all documented concepts with brief descriptions:

untangle::task_callback_for	
Can callback_t serve as the completion callback of a task returning result_t?	29
untangle::testable_for_emptyiness	
Can callable_t say whether it is empty?	30

Chapter 5

Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::actuator< action_t >	
An actuator is a functor that can trigger a dynamic list of actions (of type std::function<...>)	
	31
untangle::invalid_action	
Invalid action exception	46
untangle::actuator< action_t >::no_callback	47

Chapter 6

File Index

6.1 File List

Here is a list of all documented files with brief descriptions:

actuator.hpp	49
--	----

Chapter 7

Topic Documentation

7.1 namespace untangle: functions

Functions

- `template<typename action_t, typename... Actions>`
`actuator< action_t > untangle::connect (action_t &A1, Actions &&... An)`
Creates an actuator holding an initial list of actions.
- `template<typename action_t, typename... Actions>`
`actuator< action_t > untangle::connect (std::type_identity_t< action_t > &&A1, Actions &&... An)`
Creates an actuator owning an initial list of actions.
- `template<typename action_t, typename key_t, typename... Actions>`
`actuator< action_t > untangle::connect (std::pair< key_t, action_t * > A1, Actions &&... An)`
Creates an actuator holding an initial map of named actions.
- `template<typename action_t, typename key_t, typename value_t, typename... Actions>`
requires (!std::is_same_v<value_t, action_t*>)
`actuator< action_t > untangle::connect (std::pair< key_t, value_t > A1, Actions &&... An)`
Creates an actuator owning an initial map of named actions.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵
T>::type>>`
`action_t untangle::bind (const std::shared_ptr< class_t > &obj, T class_t::*method)`
Binding to a class function member.
- `template<typename class_t, typename R, typename... Args>`
`std::function< R(Args...)> untangle::bind (const std::shared_ptr< class_t > &obj, std::function<
R(Args...)> class_t::*field)`
Binding to a funcref field: a std::function data member, called like a method.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵
T>::type>>`
`action_t untangle::bind (class_t *obj, T class_t::*method)`
Binding to a class method.

7.1.1 Detailed Description

7.1.2 Function Documentation

7.1.2.1 `connect()` [1/4]

```
template<typename action_t, typename... Actions>
actuator< action_t > untangle::connect (
    action_t & A1,
    Actions &&... An)
```

Creates an actuator holding an initial list of actions.

Remarks

Ownership: an action passed as an lvalue is one the caller owns, and the actuator only points at it – it must outlive the actuator. An action passed as an rvalue, an anonymous lambda above all, is moved into the actuator instead and needs no named variable at all. The two can be mixed in one call. See the ownership convention on [actuator](#).

Warning

A call made only of anonymous lambdas can not deduce `action_t`. A lambda has its own closure type and is not a `std::function` until something converts it, so there is nothing in such a call to deduce the action signature from, and it has to be named:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    [](int angle) { ... }, [](int angle) { ... });
```

Warning

Naming the type of the variable the result is assigned to does not supply it:

```
// still ill formed -- "no matching function for call to connect"
untangle::actuator<std::function<void(int)>> actuator_rotate =
    untangle::connect([](int angle) { ... });
```

Template arguments are deduced from the call arguments alone, never from what the returned value is assigned to, so the explicit argument belongs on [connect\(\)](#) itself. One named action anywhere in the call deduces `action_t` for the whole of it, and the anonymous lambdas beside it then need nothing.

Parameters

A1	- The first action. It is specified as <code>std::function<...></code> .
An	- Any number of further actions, each convertible to the type of A1.

Returns

An [actuator](#).

Example:

```
void rotate_shapes(const std::vector<shape*>& shapes, int angle) {
    for (const auto& s : shapes) {
        s->rotate(angle);
    }
}

auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate(20);

// connect() deduces the action type from its arguments. A lambda has its own closure type
// and is not a std::function until something converts it, so a call made only of anonymous
// lambdas has nothing to deduce from and the signature has to be named on connect()
// itself. Naming the type of the variable assigned to would not supply it: template
// arguments are never deduced from what the returned value is assigned to.
auto actuator_scale = untangle::connect<std::function<int(int)>>([&](int v) { return v * 2; },
    [&](int v) { return v * 3; });

actuator_scale(10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(20, 30));
```

7.1.2.2 connect() [2/4]

```
template<typename action_t, typename... Actions>
actuator< action_t > untangle::connect (
    std::type_identity_t< action_t > && A1,
    Actions &&... An)
```

Creates an actuator owning an initial list of actions.

This overload takes the first action by rvalue, so it is the one that accepts a leading anonymous lambda. `std::type_identity_t` makes `action_t` a non-deduced context here, which is what both keeps this overload out of the way of the lvalue one – it is not a candidate at all unless `action_t` is named – and states the requirement described by the warning on the overload taking the first action by lvalue.

Parameters

A1	- The first action, as an rvalue.
An	- Any number of further actions, owned or pointed at by value category.

Returns

An [actuator](#).

7.1.2.3 connect() [3/4]

```
template<typename action_t, typename key_t, typename... Actions>
actuator< action_t > untangle::connect (
    std::pair< key_t, action_t * > A1,
    Actions &&... An)
```

Creates an actuator holding an initial map of named actions.

Remarks

Ownership works as it does for the unnamed [connect\(\)](#) overloads: a pair holding a pointer names an action the caller owns, and a pair holding an action by value – an anonymous lambda above all – hands it to the actuator to own. The two can be mixed in one call.

Warning

The action type is deduced from the pointer in the first pair, so a call whose first pair holds an anonymous lambda has nothing to deduce it from and has to name the signature on [connect\(\)](#) itself, exactly as the unnamed overload does:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    std::make_pair("triangle", [](int angle) { ... })),
    std::make_pair("circle", [](int angle) { ... }));
```

Remarks

action_t leads the template parameter list, ahead of key_t, so that naming it – [connect](#)<std::function<...>>(…) – means the same thing on every [connect\(\)](#) overload. Both parameters are deduced from the first pair whenever it holds a pointer, so nothing has to be named here in the first place.

Parameters

A1	- The first name/action pair. Its second element is a pointer to an action.
An	- Any number of further name/action pairs, owned or pointed at according to whether they hold the action itself or a pointer to it.

Returns

An [actuator](#).

Example:

```
// A pair holding the action itself hands it to the actuator to own. The action type is
// deduced from a pointer, and there is no pointer here, so the signature has to be named
// on connect() -- as it does for the unnamed overload.
auto actuator_scale = untangle::connect<std::function<int(int)>>(
    std::make_pair("double", [](int v) { return v * 2; }),
    std::make_pair("triple", [](int v) { return v * 3; }));

actuator_scale.invoke_action("double", 10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(20));
actuator_scale.invoke_action("triple", 10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(30));
```

7.1.2.4 connect() [4/4]

```
template<typename action_t, typename key_t, typename value_t, typename... Actions>
requires (!std::is_same_v<value_t, action_t*>)
actuator< action_t > untangle::connect (
    std::pair< key_t, value_t > A1,
    Actions &&... An)
```

Creates an actuator owning an initial map of named actions.

This overload takes a first pair holding the action itself rather than a pointer to it, so it is the one that accepts a leading anonymous lambda. `action_t` is never deduced here – the requires clause is what keeps it from competing with the overload taking a pointer in the first pair over a leading pointer pair, which both would otherwise accept once `action_t` is named.

Parameters

A1	- The first name/action pair, holding the action itself.
An	- Any number of further name/action pairs.

Returns

An [actuator](#).

7.1.2.5 bind() [1/3]

```
template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<T>::<=
type>>>
action_t untangle::bind (
    const std::shared_ptr< class_t > & obj,
    T class_t::* method)
```

Binding to a class function member.

It returns a `std::function(lambda)` that wraps the function member. It may be used to provide an action for [connect\(\)](#) or [actuator::add\(\)](#).

Remarks

It requires a shared pointer to the class type. A `std::weak_ptr` to it is captured internally in a lambda, so the binding can always check whether the shared object is still alive without keeping it alive itself. Therefore, it is safe to use actions provided by this binding inside an [actuator](#), and safe for the action to outlive the caller's shared pointer.

Parameters

obj	- Class object.
-----	-----------------

method	- Pointer to function member. It is specified as <code>&<class type>::<function member></code>
--------	--

Returns

`action_t` - A `std::function` that wraps the pointer to function member.

Remarks

If the class object gets invalid, invoking this binding will throw an exception of type [invalid_action](#).

7.1.2.6 `bind()` [2/3]

```
template<typename class_t, typename R, typename... Args>
std::function< R(Args...)> untangle::bind (
    const std::shared_ptr< class_t > & obj,
    std::function< R(Args...)> class_t::* field)
```

Binding to a funcref field: a `std::function` data member, called like a method.

It holds the class object as the method overload does, through a `std::weak_ptr`, and reads the field on every call, so a later assignment to the field is what runs.

Parameters

obj	- Class object.
field	- Pointer to the <code>std::function</code> data member, as <code>&<class type>::<field></code> .

Returns

A `std::function` that calls the field.

Remarks

Invoking this binding throws [invalid_action](#) when the class object is gone or the field is empty, so an [actuator](#) drops it either way.

7.1.2.7 `bind()` [3/3]

```
template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<T>::<←
type>>>
action_t untangle::bind (
    class_t * obj,
    T class_t::* method)
```

Binding to a class method.

Attention

It is not safe to use this binding when the pointed-to object may be destroyed. A null pointer is

Parameters

obj	- Pointer to class.
method	- Pointer to function member. It is specified as <code>&<class type>::<function member></code>

Returns

action_t - A `std::function` that wraps the pointer to function member.

Chapter 8

Namespace Documentation

8.1 untangle Namespace Reference

Interface to [untangle::actuator](#) functor.

Classes

- struct [invalid_action](#)
Invalid action exception.
- struct [actuator](#)
An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

Concepts

- concept [task_callback_for](#)
Can `callback_t` serve as the completion callback of a task returning `result_t`?
- concept [testable_for_emptyiness](#)
Can `callable_t` say whether it is empty?

Typedefs

- using [task_t](#) = `std::function<void()>`
An action, its arguments and the callback it notifies, sealed into one nullary callable.

Functions

- `template<typename... Args>`
`auto last_arg (Args &&... args)`
 Get last argument of params pack;.
- `template<typename action_t, typename arg_t>`
`void connect_one (actuator< action_t > &target, arg_t &&arg)`
 Add one `connect()` argument to the actuator being built.
- `template<typename action_t, typename... Actions>`
`actuator< action_t > connect (action_t &A1, Actions &&... An)`
 Creates an actuator holding an initial list of actions.
- `template<typename action_t, typename... Actions>`
`actuator< action_t > connect (std::type_identity_t< action_t > &&A1, Actions &&... An)`
 Creates an actuator owning an initial list of actions.
- `template<typename action_t, typename entry_t>`
`void connect_named_one (actuator< action_t > &target, entry_t &&entry)`
 Add one named `connect()` argument to the actuator being built.
- `template<typename action_t, typename key_t, typename... Actions>`
`actuator< action_t > connect (std::pair< key_t, action_t * > A1, Actions &&... An)`
 Creates an actuator holding an initial map of named actions.
- `template<typename action_t, typename key_t, typename value_t, typename... Actions>`
`requires (!std::is_same_v<value_t, action_t*>)`
`actuator< action_t > connect (std::pair< key_t, value_t > A1, Actions &&... An)`
 Creates an actuator owning an initial map of named actions.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵`
`T>::type>>`
`action_t bind (const std::shared_ptr< class_t > &obj, T class_t::*method)`
 Binding to a class function member.
- `template<typename class_t, typename R, typename... Args>`
`std::function< R(Args...)> bind (const std::shared_ptr< class_t > &obj, std::function<`
`R(Args...)> class_t::*field)`
 Binding to a funcref field: a `std::function` data member, called like a method.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵`
`T>::type>>`
`action_t bind (class_t *obj, T class_t::*method)`
 Binding to a class method.
- `template<typename action_t, typename... Args>`
`task_t bind_task (action_t action, Args &&... args)`
 Binds an action to its arguments and to the callback it must notify, which comes last.

8.1.1 Detailed Description

Interface to `untangle::actuator` functor.

8.1.2 Typedef Documentation

8.1.2.1 task_t

using `untangle::task_t = std::function<void()>`

An action, its arguments and the callback it notifies, sealed into one nullary callable.

What `bind_task()` builds, `actuator::add_task()` holds and `actuator::call_tasks()` fires.

Remarks

An action is a subscription; a task is a one-shot: it carries its own arguments, is fired once and reports to its own callback – so it is held by value, and `call_tasks()` consumes it.

Running it is what reports, and it returns nothing: the result goes to the callback and nowhere else, so there is nothing to read back and no part of it to reach from outside.

It names neither the argument types nor the result type, so one container holds tasks built from completely different actions. Anything that can be a `std::function<void()>` is a task.

8.1.3 Function Documentation

8.1.3.1 `last_arg()`

```
template<typename... Args>
auto untangle::last_arg (
    Args &&... args)
```

Get last argument of params pack;.

Template Parameters

Args	params template types
------	-----------------------

Parameters

args	params
------	--------

Returns

last param value in case it exist, otherwise 0.

8.1.3.2 `connect_one()`

```
template<typename action_t, typename arg_t>
void untangle::connect_one (
    actuator< action_t > &target,
    arg_t && arg)
```

Add one **connect()** argument to the actuator being built.

An argument that is an lvalue of the action type is one the caller owns: only its address is stored, and it has to outlive the actuator. Anything else – an anonymous lambda, a temporary `std::function`, a named lambda that is not an `action_t` yet – has no owner to outlive the actuator, so it is converted to an action and moved into it.

Remarks

Empty actions are dropped rather than stored, which is what **connect()** has always done with them. Dropping an empty owned action before it is stored is what keeps **actuator::owned** free of entries nothing points at.

Template Parameters

<code>action_↔ _t</code>	Action type of the actuator being built.
<code>arg_t</code>	Deduced type of the argument, carrying its value category.

Parameters

<code>target</code>	- The actuator being built.
<code>arg</code>	- One connect() argument.

8.1.3.3 `connect_named_one()`

```
template<typename action_t, typename entry_t>
void untangle::connect_named_one (
    actuator< action_t > & target,
    entry_t && entry)
```

Add one named [connect\(\)](#) argument to the actuator being built.

It is [connect_one\(\)](#) for the named overloads: the second element of the pair decides ownership. A pointer to an action is one the caller owns and is only stored; anything else – an anonymous lambda, a temporary `std::function` – is converted to an action and moved into the actuator.

Remarks

A null pointer is dropped rather than stored, which is what the named [connect\(\)](#) has always done with one. An owned action that is empty is dropped for the same reason the unnamed [connect_one\(\)](#) drops one: nothing would ever point at it.

Template Parameters

<code>action_↔ _t</code>	Action type of the actuator being built.
<code>entry_t</code>	Deduced type of the pair, carrying its value category.

Parameters

<code>target</code>	- The actuator being built.
<code>entry</code>	- One name/action pair.

8.1.3.4 `bind_task()`

```
template<typename action_t, typename... Args>
task_t untangle::bind_task (
    action_t action,
    Args &&... args)
```

Binds an action to its arguments and to the callback it must notify, which comes last.

Running the `task_t` it returns invokes the action and then hands the callback what it returned. The arguments are copied here, while the caller still holds them, and handed to the action as the task's own lvalues – a task runs later than it was built.

Remarks

The callback is the last argument, by position, not recognised by its type as an action's is: a pack cannot be followed by a deducible parameter, so it is split off here and callers above just forward (action, args...). `task_callback_for` only asks whether it can serve, which is what lets it be required. It is not forwarded to the action, which therefore needs no callback parameter of its own – and a void action is a task like any other.

Attention

Each argument is copied, so a move-only one does not compile. The result is not: it is moved into the callback, which may take it by rvalue reference – see `task_callback_for`.

An empty callback or an empty action builds an empty task, which `actuator::add_task()` refuses. Only a callable that can be tested for emptiness is tested, so a bare lambda is never refused.

Template Parameters

<code>action_t</code>	Type of the action; any callable naming a <code>result_type</code> will do.
<code>Args</code>	Types of the arguments to bind, of which the last is the callback.

Parameters

<code>action</code>	- The action to bind.
<code>args</code>	- The arguments to bind to action, followed by the callback to notify.

Returns

The task, or an empty one – see above.

Chapter 9

Concept Documentation

9.1 `untangle::task_callback_for` Concept Reference

Can `callback_t` serve as the completion callback of a task returning `result_t`?

```
#include <actuator.hpp>
```

9.1.1 Concept definition

```
template<typename callback_t, typename result_t>
concept task_callback_for = requires(callback_t& c, result_t&& r) {
    requires std::is_void_v<decltype(c(std::forward<result_t>(r)))>;
} || (std::is_void_v<result_t> && requires(callback_t& c) {
    requires std::is_void_v<decltype(c())>;
})
```

9.1.2 Detailed Description

Can `callback_t` serve as the completion callback of a task returning `result_t`?

It asks only whether it can serve, never whether it was meant as one: `bind_task()` takes the callback by position. A void result reports finished, so its callback takes no argument.

Remarks

The result is handed over as an rvalue, because a task keeps nothing of it: a callback may take it by value, by const reference or by rvalue reference, and a move-only result is reported like any other. A callback taking a non-const lvalue reference cannot be handed it.

Attention

Both disjuncts are load-bearing: the first yields false for a void result rather than failing hard (`void&&` in a `requires`-parameter list is a substitution failure in the immediate context), and the second's `std::is_void_v` guard keeps a `std::function<void()>` from satisfying this for any result type.

Template Parameters

callback↵ _t	Type of the callback.
result_t	Result type of the task it reports on.

9.2 untangle::testable_for_emptyiness Concept Reference

Can callable_t say whether it is empty?

```
#include <actuator.hpp>
```

9.2.1 Concept definition

```
template<typename callable_t>
concept testable_for_emptyiness =
    std::is_pointer_v<callable_t> || requires(const callable_t& c) { c.operator bool(); }
```

9.2.2 Detailed Description

Can callable_t say whether it is empty?

bind_task() refuses an empty action or an empty callback, and has to decide first whether that question can be put at all. A std::function answers it with operator bool, and a pointer with its own value.

Attention

A lambda is not asked, and asking anyway is worse than not asking. A captureless one converts implicitly to a function pointer, so a plain static_cast<bool> test compiles and answers on its behalf – always true, because that conversion never yields null. The test is dead for every lambda that reaches it, and MSVC reports the conversion it rests on as a truncation (C4305). Asking for operator bool by name is what leaves a lambda out.

Template Parameters

callable↵ _t	Type of the action or the callback.
-----------------	-------------------------------------

Chapter 10

Class Documentation

10.1 untangle::actuator< action_t > Struct Template Reference

An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

```
#include <actuator.hpp>
```

Classes

- struct [no_callback](#)

Public Types

- using [actions_t](#) = `std::list<action_t*>`
Actions container type.
- using [tasks_t](#) = `std::deque<task_t>`
Tasks container type.
- using [results_t](#) = `std::vector<typename result_t::type>`
Results container type.
- using [errors_t](#) = `std::vector<std::exception_ptr>`
Errors container type.

Public Member Functions

- [actuator](#) (const actuator &other)
Copy constructor.
- actuator & [operator=](#) (const actuator &other)
Assignment operator.
- action_t [type](#) () const
Helper method to access the action type of this object.
- bool [reset](#) ()
Remove all actions and tasks, and any stored results and errors.
- void [copy_from](#) (const actuator &other)
Copy another actuator, re-pointing the copied actions at this object's storage.
- void [release_owned](#) (const action_t *action)

- Destroy an owned action that nothing points at any more.
- `template<typename last_t>`
`void invoke_callback (last_t &last)`
 Invoke the trailing callback argument, if the invocation has one.
- `template<typename... Args>`
`void operator() (Args &&... args)`
 The call operator.
- `template<typename... Args>`
`void invoke_action (const std::string &name, Args &&... args)`
 Invokes one single action associated with a key.
- `void call_tasks ()`
 Fires every task held, in the order `add_task()` took them, and empties the list.
- `void add (action_t *action)`
 Add an action to the actions list.
- `action_t * add (action_t &&action)`
 Add an action the actuator owns.
- `void add (const std::string &name, action_t *action)`
 Add action to the actions map associated with a name.
- `action_t * add (const std::string &name, action_t &&action)`
 Add an action the actuator owns, associated with a name.
- `bool add_task (task_t &&task)`
 Adds a task, to be fired by `call_tasks()`.
- `bool remove (const action_t *action)`
 Remove an action from the actions list.
- `bool remove (const std::string &name)`
 Remove an action from actions map.
- `bool is_connected () const`
 Check if this actuator is "connected" with other actions.
- `bool has_tasks () const`
 Is there a task waiting to be fired?
- `bool is_dispatching () const`
 Whether an action is being called right now, so a removal would be refused.
- `bool has_action (const std::string &name) const`
 Check if there is certain named action.

Static Public Member Functions

- `template<typename... Args>`
`static auto take_callback (Args &... args)`
 Copies the trailing callback out of args, or nothing when the last argument is none.

Public Attributes

- `actions_t` actions
 Actions list.
- `actions_map_t` actions_map
 Named actions map.
- `results_t` results
 Actions return values list.
- `errors_t` errors

What the actions threw. See [errors_t](#).

- [tasks_t tasks](#)

Tasks waiting to be fired, in the order [add_task\(\)](#) took them.

- `std::list< action_t >` [owned](#)

Actions this actuator owns, as added by [add\(action_t&&\)](#).

- `std::size_t` [dispatching](#) = 0

10.1.1 Detailed Description

```
template<typename action_t>
struct untangle::actuator< action_t >
```

An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

Remarks

An actuator object can be constructed with an initial list of actions by [connect\(\)](#).

Callback convention: when the last argument of an invocation can be called with the action return type `R` and returns nothing, it is not passed on as a plain argument alone – it is also treated as a completion callback. The action is invoked with the full argument list as usual, and the callback is then invoked with the action return value. Any void returning callable qualifies: a `std::function<void(R)>`, a lambda taking `R`, a function pointer. It applies to [operator\(\)\(\)](#) and to [invoke_action\(\)](#) alike. Actions returning void have no result to report, so no callback is invoked for them.

Warning

The return type is what separates a callback from an argument the action means to consume itself. A trailing callable that returns a value – a transform, a comparator – is left alone, and by the same rule a callback written to return something is silently not invoked. A callback returns nothing.

Remarks

Argument convention: an action must not take ownership of the arguments it is invoked with. [operator\(\)\(\)](#) forwards one argument pack to every action in the list, and an rvalue can be moved from only once, so an action that consumes an argument leaves the actions after it holding a moved-from object – an empty `std::function` for a callback a later action means to call itself. The arguments are forwarded rather than copied so that a single action pays nothing for the broadcast; what that buys has to be respected by the actions.

Warning

An action does not have to move an argument in its body to consume it: a parameter taken by value is move constructed from an rvalue by the call itself. Take arguments by reference or const reference, or invoke with lvalues, whenever more than one action is connected. The behaviour is otherwise unpredictable, and it is the caller and the action signatures together that decide it – an actuator cannot detect the violation.

Remarks

`invoke_action()` invokes one single action, so nothing follows it and the convention does not constrain it.

Failure convention: an action that throws does not stop the invocation. What it threw is stored in `actuator::errors`, one `std::exception_ptr` per failure, and the actions after it still run. An `untangle::invalid_action` means a dead binding, so that action is dropped as well; anything else leaves the action in place. Nothing is printed and nothing escapes the call - the caller reads `actuator::errors` and decides. A failed action pushes no result, so `actuator::results` then holds fewer entries than there are actions.

Ownership convention: an actuator normally does not own its actions. It stores pointers to `std::function` objects the caller keeps alive, and those have to outlive it. `add(action_t&&)` is the exception - it moves the action into `actuator::owned` and points at the stored copy - which is what lets an anonymous lambda be an action, with no named variable to keep around. The handle it returns is the only way to `remove()` such an action afterwards.

Re-entrancy convention: nothing is removed while the actuator is dispatching - while `operator()()` or `invoke_action()` is calling an action. Removal destroys an owned action at once, so the action running, or the loop calling it, would be left on freed memory. Both `remove()` overloads and `reset()` refuse it and return false; remove once the call has returned. Adding during a dispatch is fine: an added action is called in the same pass.

Template Parameters

<code>action_t</code>	Action type. It is specified as <code>std::function<...></code> .
-----------------------	---

10.1.2 Member Typedef Documentation

10.1.2.1 `actions_t`

```
template<typename action_t>
using untangle::actuator< action_t >::actions_t = std::list<action_t*>
```

Actions container type.

Remarks

The elements stored are of pointer type, that is required to implement the `remove()` operation. `std::function` supports only equality operator for nullptr (two `std::function(s)` can not compare).

10.1.2.2 `tasks_t`

```
template<typename action_t>
using untangle::actuator< action_t >::tasks_t = std::deque<task_t>
```

Tasks container type.

Remarks

The elements are values, not pointers: identity is what `remove()` needs of an action, and no one removes a single task - `call_tasks()` consumes the whole list.

A `std::deque`, which stores its tasks in blocks: a queue adds one per submission, and a `std::list` paid an allocation for each. Nothing keeps a reference into it - `call_tasks()` takes the whole list before it fires a task. It names no result type, because `untangle::task_t` does not.

10.1.2.3 results_t

```
template<typename action_t>
using untangle::actuator< action_t >::results_t = std::vector<typename result_t::type>
```

Results container type.

It holds the return values of the actions that have a non-void return type. Upon the actuator invocation, the returns can be extracted from [results](#).

Remarks

One entry per action that returned, in invocation order. An action that threw leaves none, so results[i] is the i-th action's return only when [actuator::errors](#) is empty.

10.1.2.4 errors_t

```
template<typename action_t>
using untangle::actuator< action_t >::errors_t = std::vector<std::exception_ptr>
```

Errors container type.

It holds what the actions threw, as std::exception_ptr, in the order they were invoked.

10.1.3 Constructor & Destructor Documentation

10.1.3.1 actuator()

```
template<typename action_t>
untangle::actuator< action_t >::actuator (
    const actuator< action_t > & other) [inline]
```

Copy constructor.

Remarks

It can not be defaulted, see [copy_from\(\)](#).

10.1.4 Member Function Documentation

10.1.4.1 operator=()

```
template<typename action_t>
actuator & untangle::actuator< action_t >::operator= (
    const actuator< action_t > & other) [inline]
```

Assignment operator.

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
untangle::actuator<decltype(actuator_rotate.type())> actuator_rotate_1 = actuator_rotate;
actuator_rotate_1(20);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

10.1.4.2 type()

```
template<typename action_t>
action_t untangle::actuator< action_t >::type () const [inline]
```

Helper method to access the action type of this object.

Note

Intended to be used in expressions by `decltype(<actuator instance>.type())`

Returns

`action_t`

10.1.4.3 reset()

```
template<typename action_t>
bool untangle::actuator< action_t >::reset () [inline]
```

Remove all actions and tasks, and any stored results and errors.

After this call the actuator is empty: `actuator::is_connected()` and `actuator::has_tasks()` return false. The actions it owns are destroyed, so every handle returned by `add(action_t&&)` is dangling afterwards; the actions it merely points at are left untouched.

Returns

true - emptied. false - refused during a dispatch, and nothing changed: see the re-entrancy convention on [actuator](#).

10.1.4.4 copy_from()

```
template<typename action_t>
void untangle::actuator< action_t >::copy_from (
    const actuator< action_t > & other) [inline]
```

Copy another actuator, re-pointing the copied actions at this object's storage.

Remarks

A defaulted copy is wrong as soon as an actuator owns anything: it duplicates [actuator::owned](#) but leaves [actuator::actions](#) pointing into other's copy of it, so the new object dangles the moment other is destroyed. The owned actions are copied first, then each pointer into other::owned is translated to the matching element here.

A pointer to an action the caller owns is external to both actuators and is copied unchanged: it is not in either [actuator::owned](#), so it is not in the translation table and is left as it is.

Parameters

other	- The actuator to copy. Self copy is the caller's to exclude.
-------	---

10.1.4.5 release_owned()

```
template<typename action_t>
void untangle::actuator< action_t >::release_owned (
    const action_t * action) [inline]
```

Destroy an owned action that nothing points at any more.

Remarks

Removing an action only drops a pointer to it. When the actuator owns that action the std::function itself lives in [actuator::owned](#) and has to go too, or a loop of [add\(action_t&&\)](#) and [remove\(\)](#) grows [actuator::owned](#) without bound.

The action is kept while anything still refers to it: one handle can be added to both the list and the map, and removing it from one must not leave the other pointing at a destroyed object. An action the actuator does not own is not in [actuator::owned](#), so nothing is found to erase and the caller's object is left alone.

Parameters

action	- The action just removed. A null pointer is accepted and matches nothing.
--------	--

10.1.4.6 invoke_callback()

```
template<typename action_t>
template<typename last_t>
void untangle::actuator< action_t >::invoke_callback (
    last_t & last) [inline]
```

Invoke the trailing callback argument, if the invocation has one.

Does nothing unless last_t can be called with the action return type and returns nothing, so any other trailing argument is left alone. The test is on what the type can do, not on what it is, so a plain lambda qualifies as well as a std::function. See the callback convention on [actuator](#).

Remarks

The single condition does the work of two tests: (1) whether last can be called with the action return type at all, and (2) whether that call returns nothing. Spelled out as traits they need two nested if constexpr:

```
// (1)
if constexpr (std::is_invocable_v<last_t&, typename result_t::type>) {
    // (2)
    if constexpr (std::is_void_v<std::invoke_result_t<last_t&, typename result_t::type>») {
        last(results.back());
    }
}
```

Both are folded into the one expression instead, each in a different part of it:

```
requires { requires std::is_void_v<decltype( last(results.back()) )>; }
// \underline{(2)} \underline{(1)}
```

Remarks

(1) Invocability comes for free. To form `decltype(last(results.back()))` the compiler must first form the call `last(...)`. If `last` is an int, that expression does not exist – and inside a `requires-expression` a substitution failure yields `false` instead of being ill formed, the same property concepts are built on. So invocability is never tested separately: it is a precondition of the return type question, and failing it is what makes the whole condition false.

(2) The return type test is the inner `requires`, and the doubled keyword is not a typo. `requires { expr; }` asks only whether `expr` is well formed, and `std::is_void_v<...>` is well formed when it is `false` too – that spelling would let a value returning callable through. The second keyword makes it a nested requirement: a condition that has to hold, not merely compile.

The trait spelling has to nest, which is part of why it is not the one used here: the two `if constexpr` cannot be joined with `&&`. `&&` short circuits evaluation, not instantiation, so `std::invoke_result_t` is formed whatever the first trait says, and for a type that is not invocable at all it has no `::type` – a hard error rather than a false result.

The caller must copy the callback out of the argument pack before invoking the action: an action taking it by value moves from the caller's `std::function`, which would leave this an empty function to call.

Template Parameters

<code>last_↵ _t</code>	Type of what take_callback() returned.
----------------------------	--

Parameters

<code>last</code>	- The callback, copied before the action was invoked, or a no_callback .
-------------------	--

10.1.4.7 `take_callback()`

```
template<typename action_t>
template<typename... Args>
auto untangle::actuator< action_t >::take_callback (
    Args &... args)  [inline], [static]
```

Copies the trailing callback out of `args`, or nothing when the last argument is none.

Remarks

Only a callback is copied. An ordinary last argument is never read from here, so a copy of it would be pure cost – a whole payload per call for a caller passing a large one.

The test is [invoke_callback\(\)](#)'s own: an action returning `void` reports nothing, so it has no callback whatever its last argument is.

It is not [task_callback_for](#), which hands the result over as an rvalue. The result here stays in [actuator::results](#) and is handed over as an lvalue, so a callback taking it by rvalue reference is an ordinary argument: nothing is moved out of the results.

10.1.4.8 operator>()()

```
template<typename action_t>
template<typename... Args>
void untangle::actuator< action_t >::operator() (
    Args &&... args) [inline]
```

The call operator.

Actions in the [actuator::actions](#) list are triggered by invoking the call operator.

Parameters

args	- Arguments list must match the action arity. A trailing callback is invoked once per action, with that action's return value; see the convention on actuator .
------	---

Warning

The same argument pack is forwarded to every action in the list, so the argument convention on [actuator](#) applies here in full: an action that consumes an argument leaves the actions after it with a moved-from object.

Remarks

Nothing escapes this call: what an action throws goes to [actuator::errors](#), and the actions after it still run. See the failure convention on [actuator](#).

10.1.4.9 invoke_action()

```
template<typename action_t>
template<typename... Args>
void untangle::actuator< action_t >::invoke_action (
    const std::string & name,
    Args &&... args) [inline]
```

Invokes one single action associated with a key.

An action that can not be invoked – a null pointer, an empty std::function, a binding whose object is gone – is dropped from [actuator::actions_map](#) instead, exactly as [operator>\(\)\(\)](#) drops one from [actuator::actions](#). Nothing is invoked and nothing escapes; [actuator::has_action\(\)](#) reports it gone afterwards.

Remarks

What the action throws is stored in [actuator::errors](#), as it is for [operator>\(\)\(\)](#). See the failure convention on [actuator](#).

Parameters

name	- Key associated with the action. Invoking a key that is not in the map does nothing.
args	- Arguments list must match the action arity. A trailing callback is invoked with the action's return value; see the convention on actuator .

10.1.4.10 `call_tasks()`

```
template<typename action_t>
void untangle::actuator< action_t >::call_tasks () [inline]
```

Fires every task held, in the order `add_task()` took them, and empties the list.

Each task notifies its own callback as it runs, and its result goes there alone – never to `actuator::results`. It does not fire the actions, as `operator()()` does not fire the tasks.

Remarks

Errors are appended to `actuator::errors`, which `operator()()` clears instead – so `one(); one.call_tasks();` reports both kinds together, and the actions go first.

Attention

Finished does not mean failed. A task that throws does not stop the pass and is not notified; a callback that throws lands in `actuator::errors` the same way. The task is consumed either way, so a caller waiting on the callback alone must read the errors.

10.1.4.11 `add()` [1/4]

```
template<typename action_t>
void untangle::actuator< action_t >::add (
    action_t * action) [inline]
```

Add an action to the actions list.

Parameters

action	- Action to be added.
--------	-----------------------

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).Times(2).WillRepeatedly(testing::Return());
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate.add(&action1);
actuator_rotate(20);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

10.1.4.12 add() [2/4]

```
template<typename action_t>
action_t * untangle::actuator< action_t >::add (
    action_t && action)    [inline]
```

Add an action the actuator owns.

The action is moved into `actuator::owned`, so it needs no named variable outliving the actuator: an anonymous lambda can be passed straight in, and the call converts it to `action_t`. See the ownership convention on `actuator`.

Parameters

action	- Action to be added. It is taken by value and moved from.
--------	--

Returns

`action_t*` - A handle to the stored action, to pass to `remove()`. It stays valid until that action is removed or the actuator is destroyed. A copy of the actuator owns its own copy of the action and has its own handle to it.

Example:

```
// An anonymous lambda has no named variable to outlive the actuator, so the actuator takes
// ownership of it: it is moved into actuator::owned and the list points at the stored copy.
untangle::actuator<std::function<int(int)>> actuator_scale;
actuator_scale.add([](int v) { return v * 2; });
actuator_scale.add([](int v) { return v * 3; });

actuator_scale(10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(20, 30));
```

10.1.4.13 add() [3/4]

```
template<typename action_t>
void untangle::actuator< action_t >::add (
    const std::string & name,
    action_t * action)    [inline]
```

Add action to the actions map associated with a name.

Parameters

name	- Name of the action.
action	- Action to be added.

10.1.4.14 add() [4/4]

```
template<typename action_t>
action_t * untangle::actuator< action_t >::add (
    const std::string & name,
    action_t && action)    [inline]
```

Add action to the actions map associated with a name.

Parameters

name	- Name of the action.
action	- Action to be added. It is taken by value and moved from.

Returns

action_t* - A handle to the stored action, as `add(action_t&&)` returns, or nullptr when name is already taken. The name already in the map keeps the action it has, as it does for `add(const std::string&, action_t*)`, and the action offered here is dropped rather than left ownerless in `actuator::owned`.

10.1.4.15 add_task()

```
template<typename action_t>
bool untangle::actuator< action_t >::add_task (
    task_t && task) [inline]
```

Adds a task, to be fired by `call_tasks()`.

It is stored by value: an actuator owns every task it holds, unlike the actions it merely points at. Anything that can be an `untangle::task_t` is taken, from `bind_task()` or not. There is no counterpart to `remove()` and no named form – `call_tasks()` consumes the list, and tasks fire as a batch, in the order they were added.

Attention

It refuses an empty task, and the refusal is the whole report: nothing is thrown and nothing is stored. It lands while the caller is still on the stack, rather than surfacing from `call_tasks()` as a `std::bad_function_call` read as the failure of a task that had in fact run.

Parameters

task	- The task to add. Taken by value and moved from; a refused one is dropped.
------	---

Returns

true - taken, and `call_tasks()` will fire it.
false - refused: it is empty, so it can neither run nor report.

10.1.4.16 remove() [1/2]

```
template<typename action_t>
bool untangle::actuator< action_t >::remove (
    const action_t * action) [inline]
```

Remove an action from the actions list.

Generated by Doxygen

An invalid action (empty `std::function`) is implicitly removed when `operator>()` is invoked.

An action the actuator owns is destroyed by this call, so the handle to it is dangling afterwards. See

Parameters

action	- Action to be removed. For an owned action this is the handle returned by <code>add(action_t&&)</code> .
--------	---

Returns

true - removed. false - refused during a dispatch (see the re-entrancy convention on [actuator](#)), or not in the actions list.

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).Times(0);
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate.remove(&action1);
actuator_rotate(50);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

10.1.4.17 remove() [2/2]

```
template<typename action_t>
bool untangle::actuator< action_t >::remove (
    const std::string & name) [inline]
```

Remove an action from actions map.

An action the actuator owns is destroyed by this call, so the handle to it is dangling afterwards. See [release_owned\(\)](#).

Parameters

name	- Name of the action to remove.
------	---------------------------------

Returns

true - removed. false - refused during a dispatch (see the re-entrancy convention on [actuator](#)), or no action has that name.

10.1.4.18 `is_connected()`

```
template<typename action_t>
bool untangle::actuator< action_t >::is_connected () const [inline]
```

Check if this actuator is "connected" with other actions.

Remarks

It answers for the actions and for nothing else. Tasks are not actions and do not make an actuator connected; ask `has_tasks()` about those. Kept this way deliberately when tasks arrived, because a caller that will never hold a task – an attachment, a poll – reads this and must keep reading the answer it always read.

Returns

true - if the `actuator::actions` list is not empty.
false - if the `actuator::actions` list is empty.

10.1.4.19 `has_tasks()`

```
template<typename action_t>
bool untangle::actuator< action_t >::has_tasks () const [inline]
```

Is there a task waiting to be fired?

The counterpart to `is_connected()`, and the two answer different questions: this one is about `actuator::tasks`, that one about `actuator::actions`. An actuator can hold either, both, or neither.

Remarks

It is what a queue asks to decide whether its worker still has work. Reading `is_connected()` for that would report a batch of tasks as nothing to do, and the work would sit unfired.

A refused task is not a waiting one: `add_task()` answering false means the task was dropped, so this stays false.

Returns

true - a task is waiting; `call_tasks()` has something to fire.
false - no task is waiting. True again after `call_tasks()`, which consumes them.

10.1.4.20 `has_action()`

```
template<typename action_t>
bool untangle::actuator< action_t >::has_action (
    const std::string & name) const [inline]
```

Check if there is certain named action.

Parameters

name	- Name associated with the action.
------	------------------------------------

Returns

true - if name can be found in [actuator::actions_map](#)
false - if name can not be found in [actuator::actions_map](#)

10.1.5 Member Data Documentation

10.1.5.1 tasks

```
template<typename action_t>
tasks_t untangle::actuator< action_t >::tasks
```

Tasks waiting to be fired, in the order [add_task\(\)](#) took them.

Remarks

Held beside [actions](#) and sharing nothing with them: [operator>\(\)\(\)](#) fires the actions and leaves these alone, [call_tasks\(\)](#) fires these and leaves the actions alone.

10.1.5.2 owned

```
template<typename action_t>
std::list<action_t> untangle::actuator< action_t >::owned
```

Actions this actuator owns, as added by [add\(action_t&&\)](#).

Remarks

It is a `std::list` because inserting into one never invalidates the address of an element already in it, so every handle already handed out stays valid as further actions are added. A `std::vector` would reallocate and dangle all of them at once.

10.1.5.3 dispatching

```
template<typename action_t>
std::size_t untangle::actuator< action_t >::dispatching = 0
```

How many [operator>\(\)\(\)](#) or [invoke_action\(\)](#) calls are calling an action: more than one when an action dispatches this actuator again. See the re-entrancy convention.

The documentation for this struct was generated from the following file:

- `actuator.hpp`

10.2 untangle::invalid_action Struct Reference

Invalid action exception.

```
#include <actuator.hpp>
```

Public Member Functions

- [invalid_action](#) (std::string text)
Construct a new invalid action object.
- const char * [what](#) () const noexcept override
The message text describing the reason of this exception.

10.2.1 Detailed Description

Invalid action exception.

Remarks

An action may be provided as a binding to a class function member, by using [untangle::bind\(\)](#). When the class object gets invalid, invoking the action will raise an exception to this type.

10.2.2 Constructor & Destructor Documentation

10.2.2.1 invalid_action()

```
untangle::invalid_action::invalid_action (  
    std::string text) [inline], [explicit]
```

Construct a new invalid action object.

Parameters

text	- A message text, describing the reason of this exception.
------	--

10.2.3 Member Function Documentation

10.2.3.1 what()

```
const char * untangle::invalid_action::what () const [inline], [override], [noexcept]
```

The message text describing the reason of this exception.

Returns

const char* - The message text.

The documentation for this struct was generated from the following file:

- actuator.hpp

10.3 untangle::actuator< action_t >::no_callback Struct Reference

```
#include <actuator.hpp>
```

10.3.1 Detailed Description

```
template<typename action_t>  
struct untangle::actuator< action_t >::no_callback
```

What [take_callback\(\)](#) holds when there is no callback: [invoke_callback\(\)](#) cannot call it, so it does nothing.

The documentation for this struct was generated from the following file:

- actuator.hpp

Chapter 11

File Documentation

11.1 actuator.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #ifndef UNTANGLE_ACTUATOR_ACTUATOR_HPP
00007 #define UNTANGLE_ACTUATOR_ACTUATOR_HPP
00008
00009 #include <cstdlib>
00010 #include <deque>
00011 #include <exception>
00012 #include <functional>
00013 #include <list>
00014 #include <map>
00015 #include <memory>
00016 #include <string>
00017 #include <tuple>
00018 #include <type_traits>
00019 #include <utility>
00020 #include <vector>
00021
00022 namespace untangle {
00023 // exception
00032 struct invalid_action : std::exception {
00038     explicit invalid_action(std::string text) : message(std::move(text)) {}
00039
00045     const char* what() const noexcept override { return message.c_str(); }
00046
00047 private:
00048     std::string message;
00049 };
00050
00058 template <typename... Args>
00059 auto last_arg(Args&&... args) {
00060     if constexpr (sizeof...(Args) <= 0) {
00061         return 0;
00062     } else {
00063         return std::get<sizeof...(Args) - 1>(std::forward_as_tuple(args...));
00064     }
00065 }
00066
00085 template <typename callback_t, typename result_t>
00086 concept task_callback_for = requires(callback_t& c, result_t&& r) {
00087     requires std::is_void_v<decltype(c(std::forward<result_t>(r)))>;
00088 } || (std::is_void_v<result_t> && requires(callback_t& c) {
00089     requires std::is_void_v<decltype(c())>;
00090     });
00091
00107 template <typename callable_t>
00108 concept testable_for_emptyness =
00109     std::is_pointer_v<callable_t> || requires(const callable_t& c) { c.operator bool(); };
00110
00126 using task_t = std::function<void()>;
00127
00186 template <typename action_t>
00187 struct actuator final {
00195     using actions_t = std::list<action_t*>;
00196     using actions_map_t = std::map<std::string, action_t*>;
00208     using tasks_t = std::deque<task_t>;
00209     using result_t = std::conditional<std::is_void<typename action_t::result_type>::value, int,
```

```

00210         typename action_t::result_type>;
00220 using results_t = std::vector<typename result_t::type>;
00226 using errors_t = std::vector<std::exception_ptr>;
00227
00228 actions_t actions;
00229 actions_map_t actions_map;
00230 results_t results;
00231 errors_t errors;
00232
00239 tasks_t tasks;
00240
00248 std::list<action_t> owned;
00249
00252 std::size_t dispatching = 0;
00253
00254 actuator() = default;
00260 actuator(const actuator& other) { copy_from(other); }
00261 actuator(actuator&&) noexcept = default;
00262 ~actuator() = default;
00269 actuator& operator=(const actuator& other) {
00270     if (this != &other) {
00271         copy_from(other);
00272     }
00273     return *this;
00274 }
00275 actuator& operator=(actuator&&) noexcept = default;
00276
00284 action_t type() const { return nullptr; }
00285
00297 bool reset() {
00298     if (dispatching > 0) {
00299         return false;
00300     }
00301     actions.clear();
00302     actions_map.clear();
00303     tasks.clear();
00304     results.clear();
00305     errors.clear();
00306     owned.clear();
00307     return true;
00308 }
00309
00324 void copy_from(const actuator& other) {
00325     owned = other.owned;
00326
00327     // Old address -> new address, for the owned actions only.
00328     std::map<const action_t*, action_t*> remap;
00329     auto src = other.owned.begin();
00330     auto dst = owned.begin();
00331     for (; src != other.owned.end(); ++src, ++dst) {
00332         remap.emplace(&*src, &*dst);
00333     }
00334     const auto translate = [&remap](action_t* action) {
00335         const auto it = remap.find(action);
00336         return it != remap.end() ? it->second : action;
00337     };
00338
00339     actions.clear();
00340     for (const auto& action : other.actions) {
00341         actions.push_back(translate(action));
00342     }
00343     actions_map.clear();
00344     for (const auto& entry : other.actions_map) {
00345         actions_map.emplace(entry.first, translate(entry.second));
00346     }
00347     // Named explicitly, like every member here: this function does not default, so anything it
00348     // does not copy is silently lost by every copy of an actuator.
00349     tasks = other.tasks;
00350     results = other.results;
00351     errors = other.errors;
00352 }
00353
00368 void release_owned(const action_t* action) {
00369     for (const auto& a : actions) {
00370         if (a == action) {
00371             return;
00372         }
00373     }
00374     for (const auto& entry : actions_map) {
00375         if (entry.second == action) {
00376             return;
00377         }
00378     }
00379     owned.remove_if([action](const action_t& a) { return &a == action; });
00380 }
00381
00437 template <typename last_t>

```

```

00438 void invoke_callback(last_t& last) {
00439     if constexpr (requires { requires std::is_void_v<decltype(last(results.back()))>; }) {
00440         last(results.back());
00441     }
00442 }
00443
00446 struct no_callback {};
00447
00461 template <typename... Args>
00462 static auto take_callback(Args&... args) {
00463     using result_t = typename action_t::result_type;
00464
00465     if constexpr (sizeof...(Args) == 0 || std::is_void_v<result_t>) {
00466         return no_callback{};
00467     } else {
00468         using last_t = std::decay_t<std::tuple_element_t<sizeof...(Args) - 1, std::tuple<Args...>>;
00469
00470         if constexpr (requires(last_t& c, result_t& r) { requires std::is_void_v<decltype(c(r))>; }) {
00471             return last_arg(args...);
00472         } else {
00473             return no_callback{};
00474         }
00475     }
00476 }
00477
00493 template <typename... Args>
00494 void operator()(Args&&... args) {
00495     results.clear();
00496     errors.clear();
00497
00498     // Copied up front, before any action can move the callback out of the argument pack.
00499     auto last = take_callback(args...);
00500
00501     // Dead bindings are collected here and dropped after the loop.
00502     // The actuator does not own the actions it points at, so it must never
00503     // write through action_t* into a std::function belonging to the caller.
00504     std::vector<action_t*> dead_actions;
00505
00506     ++dispatching;
00507     for (const auto& action : actions) {
00508         // A null pointer or an empty std::function can never be invoked. Calling an
00509         // empty one throws std::bad_function_call, which is not an invalid_action and
00510         // would escape this operator, so drop it instead of invoking it.
00511         if (action == nullptr || !*action) {
00512             dead_actions.push_back(action);
00513             continue;
00514         }
00515
00516         try {
00517             if constexpr (std::is_same_v<typename action_t::result_type, void>) {
00518                 (*action)(std::forward<Args>(args)...);
00519             } else {
00520                 results.push_back((*action)(std::forward<Args>(args)...));
00521                 invoke_callback(last);
00522             }
00523         } catch (const invalid_action&) {
00524             // A dead binding is not coming back: it is recorded and the action is dropped.
00525             errors.push_back(std::current_exception());
00526             dead_actions.push_back(action);
00527         } catch (...) {
00528             // Anything else is the action's own failure. It is recorded, the action is kept, and the
00529             // actions behind it still run.
00530             errors.push_back(std::current_exception());
00531         }
00532     }
00533
00534     --dispatching;
00535
00536     // Refused inside a dispatch further out; the next dispatch finds them dead again.
00537     for (const auto& dead_action : dead_actions) {
00538         remove(dead_action);
00539     }
00540 }
00541
00558 template <typename... Args>
00559 void invoke_action(const std::string& name, Args&&... args) {
00560     results.clear();
00561     errors.clear();
00562
00563     // Copied up front, before the action can move the callback out of the argument pack.
00564     auto last = take_callback(args...);
00565     const auto it = actions_map.find(name);
00566     if (it == actions_map.end()) {
00567         return;
00568     }
00569
00570     // A null pointer or an empty std::function can never be invoked, and dereferencing a

```

```

00571 // null one to try is undefined behaviour before a call is even made. Calling an empty
00572 // one throws std::bad_function_call, which is not an invalid_action and would escape
00573 // this method. Drop it instead, as operator>() does for the actions list.
00574 if (it->second == nullptr || !*it->second) {
00575     remove(name);
00576     return;
00577 }
00578
00579 ++dispatching;
00580 bool dead = false;
00581 try {
00582     if constexpr (std::is_same_v<typename action_t::result_type, void>) {
00583         (*it->second)(std::forward<Args>(args)...);
00584     } else {
00585         results.push_back((*it->second)(std::forward<Args>(args)...));
00586         invoke_callback(last);
00587     }
00588 } catch (const invalid_action&) {
00589     errors.push_back(std::current_exception());
00590     dead = true;
00591 } catch (...) {
00592     errors.push_back(std::current_exception());
00593 }
00594 --dispatching;
00595
00596 if (dead) {
00597     remove(name);
00598 }
00599 }
00600
00614 void call_tasks() {
00615     // Taken by swap, so a callback that adds a task adds it to the *next* pass. Iterating the
00616     // member instead would let a task that re-adds itself keep this loop from ever ending -- and
00617     // it is the same reason a queue built on this takes its batch by move.
00618     tasks_t firing;
00619     firing.swap(tasks);
00620
00621     for (auto& task : firing) {
00622         try {
00623             // The task notifies its own callback: bind_task() sealed the two together.
00624             task();
00625         } catch (...) {
00626             // The task's own failure, or its callback's -- the two are not told apart here, and the
00627             // attention above says why.
00628             errors.push_back(std::current_exception());
00629         }
00630     }
00631 }
00632
00641 void add(action_t* action) { actions.push_back(action); }
00642
00658 action_t* add(action_t&& action) {
00659     owned.push_back(std::move(action));
00660     actions.push_back(&owned.back());
00661     return &owned.back();
00662 }
00663
00670 void add(const std::string& name, action_t* action) { actions_map.emplace(name, action); }
00671
00682 action_t* add(const std::string& name, action_t&& action) {
00683     owned.push_back(std::move(action));
00684     const auto [it, inserted] = actions_map.emplace(name, &owned.back());
00685     if (inserted) {
00686         owned.pop_back();
00687         return nullptr;
00688     }
00689     return &owned.back();
00690 }
00691
00710 bool add_task(task_t&& task) {
00711     if (!task) {
00712         return false;
00713     }
00714
00715     tasks.push_back(std::move(task));
00716     return true;
00717 }
00718
00735 bool remove(const action_t* action) {
00736     if (dispatching > 0) {
00737         return false;
00738     }
00739     const auto removed = actions.remove_if([&action](const auto& a) { return (action == a); });
00740     release_owned(action);
00741     return removed > 0;
00742 }
00743

```

```

00754 bool remove(const std::string& name) {
00755     if (dispatching > 0) {
00756         return false;
00757     }
00758     const auto it = actions_map.find(name);
00759     if (it == actions_map.end()) {
00760         return false;
00761     }
00762     const action_t* action = it->second;
00763     actions_map.erase(it);
00764     release_owned(action);
00765     return true;
00766 }
00767
00779 bool is_connected() const { return !actions.empty() || !actions_map.empty(); }
00780
00798 bool has_tasks() const { return !tasks.empty(); }
00799
00801 bool is_dispatching() const { return dispatching > 0; }
00802
00810 bool has_action(const std::string& name) const {
00811     return actions_map.find(name) != actions_map.end();
00812 }
00813 };
00814
00832 template <typename action_t, typename arg_t>
00833 void connect_one(actuator<action_t>& target, arg_t&& arg) {
00834     if constexpr (std::is_lvalue_reference_v<arg_t&&> &&
00835         std::is_same_v<std::decay_t<arg_t>, action_t>) {
00836         if (arg != nullptr) {
00837             target.add(&arg);
00838         }
00839     } else {
00840         action_t action(std::forward<arg_t>(arg));
00841         if (action != nullptr) {
00842             target.add(std::move(action));
00843         }
00844     }
00845 }
00846
00889 template <typename action_t, typename... Actions>
00890 actuator<action_t> connect(action_t& A1, Actions&&... An) {
00891     actuator<action_t> target;
00892     connect_one(target, A1);
00893     (connect_one(target, std::forward<Actions>(An)), ...);
00894     return target;
00895 }
00896
00913 template <typename action_t, typename... Actions>
00914 actuator<action_t> connect(std::type_identity_t<action_t>&& A1, Actions&&... An) {
00915     actuator<action_t> target;
00916     connect_one(target, std::move(A1));
00917     (connect_one(target, std::forward<Actions>(An)), ...);
00918     return target;
00919 }
00920
00921 template <typename actuator_t>
00922 void remove_empty_actions(actuator_t& actuator) {
00923     std::vector<typename actuator_t::actions_map_t::const_iterator> removable_iterators;
00924     for (auto it = actuator.actions_map.begin(); it != actuator.actions_map.end(); ++it) {
00925         if (it->second == nullptr) {
00926             removable_iterators.push_back(it);
00927         }
00928     }
00929     for (auto it : removable_iterators) {
00930         actuator.actions_map.erase(it);
00931     }
00932 }
00933
00951 template <typename action_t, typename entry_t>
00952 void connect_named_one(actuator<action_t>& target, entry_t&& entry) {
00953     if constexpr (std::is_same_v<std::remove_cvref_t<decltype(entry.second)>, action_t*>) {
00954         if (entry.second != nullptr) {
00955             target.add(entry.first, entry.second);
00956         }
00957     } else {
00958         action_t action(std::forward<entry_t>(entry).second);
00959         if (action != nullptr) {
00960             target.add(entry.first, std::move(action));
00961         }
00962     }
00963 }
00964
00999 template <typename action_t, typename key_t, typename... Actions>
01000 actuator<action_t> connect(std::pair<key_t, action_t*> A1, Actions&&... An) {
01001     actuator<action_t> target;
01002     connect_named_one(target, std::move(A1));

```

```

01003 (connect_named_one(target, std::forward<Actions>(An)), ...);
01004 return target;
01005 }
01006
01023 template <typename action_t, typename key_t, typename value_t, typename... Actions>
01024 requires(!std::is_same_v<value_t, action_t*>)
01025 actuator<action_t> connect(std::pair<key_t, value_t> A1, Actions&&... An) {
01026     actuator<action_t> target;
01027     connect_named_one(target, std::move(A1));
01028     (connect_named_one(target, std::forward<Actions>(An)), ...);
01029     return target;
01030 }
01031
01032 // generic helpers to remove the const and noexcept qualifiers from a function type,
01033 // for instance const member functions
01034 template <typename T>
01035 struct function_remove_const;
01036
01037 template <typename R, typename... Args>
01038 struct function_remove_const<R(Args...)> {
01039     using type = R(Args...);
01040 };
01041
01042 template <typename R, typename... Args>
01043 struct function_remove_const<R(Args...) const> {
01044     using type = R(Args...);
01045 };
01046
01047 template <typename R, typename... Args>
01048 struct function_remove_const<R(Args...) noexcept> {
01049     using type = R(Args...);
01050 };
01051
01052 template <typename R, typename... Args>
01053 struct function_remove_const<R(Args...) const noexcept> {
01054     using type = R(Args...);
01055 };
01056
01060
01082 template <typename class_t, typename T,
01083           typename action_t = std::function<typename function_remove_const<T>::type>
01084           action_t bind(const std::shared_ptr<class_t>& obj, T class_t::* method) {
01085     return [wp = std::weak_ptr<class_t>(obj), method](auto&&... args) ->
01086         typename action_t::result_type {
01087         // lock() also keeps the object alive for the duration of the call
01088         const auto obj_ = wp.lock();
01089         if (!obj_) {
01090             // inform the actuator about dead binding
01091             throw invalid_action("bind: invalid object");
01092         }
01093         return ((*obj_).*method)(std::forward<decltype(args)>(args)...);
01094     };
01095 }
01096
01112 template <typename class_t, typename R, typename... Args>
01113 std::function<R(Args...)> bind(const std::shared_ptr<class_t>& obj,
01114                               std::function<R(Args...)> class_t::* field) {
01115     return [wp = std::weak_ptr<class_t>(obj), field](auto&&... args) -> R {
01116         // lock() also keeps the object alive for the duration of the call
01117         const auto obj_ = wp.lock();
01118         if (!obj_) {
01119             throw invalid_action("bind: invalid object");
01120         }
01121         const auto& function = (*obj_).*field;
01122         if (!function) {
01123             throw invalid_action("bind: empty field");
01124         }
01125         return function(std::forward<decltype(args)>(args)...);
01126     };
01127 }
01128
01149 template <typename class_t, typename T,
01150           typename action_t = std::function<typename function_remove_const<T>::type>
01151           action_t bind(class_t* obj, T class_t::* method) {
01152     return [obj, method](auto&&... args) -> typename action_t::result_type {
01153         if (!obj) {
01154             // inform the actuator about dead binding
01155             throw invalid_action("bind: invalid object");
01156         }
01157         return ((obj)->*method)(std::forward<decltype(args)>(args)...);
01158     };
01159 }
01160
01188 template <typename action_t, typename... Args>
01189 task_t bind_task(action_t action, Args&&... args) {
01190     using result_t = typename action_t::result_type;
01191

```

```

01192 static_assert(sizeof...(Args) > 0,
01193               "bind_task: a task must be given a callback as its last argument");
01194
01195 constexpr std::size_t last = sizeof...(Args) - 1;
01196 using callback_t = std::tuple_element_t<last, std::tuple<std::decay_t<Args>...>>;
01197
01198 static_assert(task_callback_for<callback_t, result_t>,
01199               "bind_task: the last argument must be a callback taking the task's result and "
01200               "returning nothing (void()) for a task that returns nothing");
01201
01202 // References into the caller's arguments. What is copied out of it is the task's own, below.
01203 auto pack = std::forward_as_tuple(std::forward<Args>(args)...);
01204
01205 // A task that could not do its job is built empty, and add_task() refuses it on the caller's
01206 // own stack. Only a callable that can answer is asked -- see \ref testable_for_emptyiness -- and
01207 // the answer is read contextually, so no explicit conversion is left to truncate.
01208 if constexpr (testable_for_emptyiness<callback_t>) {
01209     if (!std::get<last>(pack)) {
01210         return {};
01211     }
01212 }
01213 if constexpr (testable_for_emptyiness<action_t>) {
01214     if (!laction) {
01215         return {};
01216     }
01217 }
01218
01219 // The pack is indexed rather than unpacked, because everything but its last element is bound and
01220 // a fold cannot say "all but the last". The bound arguments are copied into one tuple rather
01221 // than into a capture pack: an init-capture pack expanded in a dependent context crashes GCC 14
01222 // (an ICE in tsubst_pack_expansion), and a tuple says the same thing in an ordinary expression.
01223 return [&<std::size_t... i>(std::index_sequence<i...>) -> task_t {
01224     return [action = std::move(action), callback = callback_t(std::get<last>(pack)),
01225           bound = std::tuple<std::decay_t<decltype(std::get<i>(pack))>...>(
01226             std::get<i>(pack)...)]() mutable -> void {
01227         if constexpr (std::is_void_v<result_t>) {
01228             std::apply(action, bound);
01229             callback();
01230         } else {
01231             // One expression, so the result is handed over as an rvalue: a result type that cannot be
01232             // copied is still a result a task can report.
01233             callback(std::apply(action, bound));
01234         }
01235     };
01236 }(std::make_index_sequence<last>{}));
01237 }
01238
01239 } // namespace untangle
01240
01241 #endif // UNTANGLE_ACTUATOR_ACTUATOR_HPP

```


Index

actions_t
 untangle::actuator< action_t >, 34
actuator, 1
 untangle::actuator< action_t >, 35
add
 untangle::actuator< action_t >, 40, 41
add_task
 untangle::actuator< action_t >, 42

bind
 namespace untangle: functions, 19, 20
bind_task
 untangle, 26

call_tasks
 untangle::actuator< action_t >, 40
connect
 namespace untangle: functions, 16, 17, 18
connect_named_one
 untangle, 26
connect_one
 untangle, 25
copy_from
 untangle::actuator< action_t >, 36

dispatching
 untangle::actuator< action_t >, 45

errors_t
 untangle::actuator< action_t >, 35

has_action
 untangle::actuator< action_t >, 44
has_tasks
 untangle::actuator< action_t >, 44

invalid_action
 untangle::invalid_action, 46
invoke_action
 untangle::actuator< action_t >, 39
invoke_callback
 untangle::actuator< action_t >, 37
is_connected
 untangle::actuator< action_t >, 43

last_arg
 untangle, 25

namespace untangle: functions, 15
 bind, 19, 20
 connect, 16, 17, 18

operator()
 untangle::actuator< action_t >, 38
operator=
 untangle::actuator< action_t >, 35
owned
 untangle::actuator< action_t >, 45

release_owned
 untangle::actuator< action_t >, 37
remove
 untangle::actuator< action_t >, 42, 43
reset
 untangle::actuator< action_t >, 36
results_t
 untangle::actuator< action_t >, 34

take_callback
 untangle::actuator< action_t >, 38
task_t
 untangle, 24
tasks
 untangle::actuator< action_t >, 45
tasks_t
 untangle::actuator< action_t >, 34
type
 untangle::actuator< action_t >, 35

untangle, 23
 bind_task, 26
 connect_named_one, 26
 connect_one, 25
 last_arg, 25
 task_t, 24
untangle::actuator< action_t >, 31
 actions_t, 34
 actuator, 35
 add, 40, 41
 add_task, 42
 call_tasks, 40
 copy_from, 36
 dispatching, 45
 errors_t, 35
 has_action, 44
 has_tasks, 44
 invoke_action, 39
 invoke_callback, 37
 is_connected, 43
 operator(), 38
 operator=, 35
 owned, 45

- release_owned, [37](#)
- remove, [42](#), [43](#)
- reset, [36](#)
- results_t, [34](#)
- take_callback, [38](#)
- tasks, [45](#)
- tasks_t, [34](#)
- type, [35](#)
- untangle::actuator< action_t >::no_callback, [47](#)
- untangle::invalid_action, [46](#)
 - invalid_action, [46](#)
 - what, [46](#)
- untangle::task_callback_for, [29](#)
- untangle::testable_for_emptiness, [30](#)
- what
 - untangle::invalid_action, [46](#)